SOURCE CODE LISTING

LedgerLens

Banking Transaction Analysis (Flask)

Live demo: banking-flask-project.vercel.app

Project files included

01	app.py	Flask application: models, routes, business rules
02	api/index.py	Vercel serverless entry point
03	vercel.json	Vercel build and routing configuration
04	requirements.txt	Python dependencies
05	supabase/schema.sql	PostgreSQL schema for Supabase
06	analysis/analyze_transactions.py	Optional pandas / NumPy / Matplotlib analysis script
07	templates/base.html	Base layout: sidebar, top bar, flash messages
08	templates/index.html	Dashboard page with Chart.js charts
09	templates/transactions.html	Transactions list, search/filter and add form
10	templates/accounts.html	Account cards and create-account form
11	static/style.css	Stylesheet

Not included: .venv, instance/ (local SQLite file) and generated files. Data in the project is synthetic.

01 · app.py

Flask application: models, routes, business rules

```python
import os
from datetime import datetime, date
from decimal import Decimal, InvalidOperation
from flask import Flask, render_template, request, redirect, url_for, flash, jsonify
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.secret_key = os.getenv('FLASK_SECRET_KEY', 'change-this-for-deployment')

# Local development uses SQLite. Set DATABASE_URL to a Supabase Postgres connection string for deployment.
database_url = os.getenv('DATABASE_URL', 'sqlite:///banking_analysis.db')
if database_url.startswith('postgres://'):
    database_url = database_url.replace('postgres://', 'postgresql://', 1)
if database_url.startswith('postgresql://') and 'sslmode=' not in database_url:
    database_url += ('&' if '?' in database_url else '?') + 'sslmode=require'
app.config['SQLALCHEMY_DATABASE_URI'] = database_url
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)

class Customer(db.Model):
    __tablename__ = 'customers'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(120), nullable=False, default='Demo Customer')
    email = db.Column(db.String(180), nullable=False, unique=True, default='demo@example.com')
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    accounts = db.relationship('Account', backref='customer', cascade='all, delete-orphan')

class Account(db.Model):
    __tablename__ = 'accounts'
    id = db.Column(db.Integer, primary_key=True)
    account_number = db.Column(db.String(32), unique=True, nullable=False)
    name = db.Column(db.String(100), nullable=False)
    account_type = db.Column(db.String(40), nullable=False, default='Savings')
    opening_balance = db.Column(db.Float, nullable=False, default=0)
    customer_id = db.Column(db.Integer, db.ForeignKey('customers.id'), nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    transactions = db.relationship('Transaction', backref='account', cascade='all, delete-orphan')

class Transaction(db.Model):
    __tablename__ = 'transactions'
    id = db.Column(db.Integer, primary_key=True)
    reference = db.Column(db.String(24), unique=True, nullable=False)
    account_id = db.Column(db.Integer, db.ForeignKey('accounts.id'), nullable=False)
    txn_type = db.Column(db.String(20), nullable=False)
    amount = db.Column(db.Float, nullable=False)
    description = db.Column(db.String(240), nullable=False, default='General transaction')
    txn_date = db.Column(db.Date, nullable=False, default=date.today)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    @property
    def is_flagged(self):
        return self.txn_type == 'Withdrawal' and self.amount >= 2000 or self.amount >= 5000

    @property
    def flag_reason(self):
        if self.amount >= 5000: return 'High-value transaction (≥ ₹5,000)'
        if self.txn_type == 'Withdrawal' and self.amount >= 2000: return 'Large withdrawal (≥ ₹2,000)'
        return ''

with app.app_context():
    db.create_all()
    if Customer.query.count() == 0:
        c = Customer(name='Demo Customer', email='demo@example.com')
        db.session.add(c); db.session.flush()
        a1 = Account(account_number='**** 1234', name='Primary Savings', account_type='Savings', opening_balance=3420, customer_id=c.id)
        a2 = Account(account_number='**** 5678', name='Everyday Checking', account_type='Checking', opening_balance=1250, customer_id=c.id)
        a3 = Account(account_number='**** 9012', name='Fixed Deposit', account_type='Fixed Deposit', opening_balance=750, customer_id=c.id)
        db.session.add_all([a1, a2, a3]); db.session.flush()
        samples = [
            (a1,'Deposit',1200,'Salary credit',date(2026,8,24)), (a2,'Withdrawal',450,'ATM withdrawal',date(2026,8,23)),
```

```
            (a1,'Transfer',300,'Transfer to checking',date(2026,8,22)), (a3,'Deposit',2000,'FD renewal',date(2026,8,21)),
            (a2,'Withdrawal',120,'Online purchase',date(2026,8,20)), (a1,'Deposit',800,'Freelance payment',date(2026,8,19)),
            (a2,'Withdrawal',250,'Grocery store',date(2026,8,18)), (a1,'Withdrawal',2500,'Large cash withdrawal',date(2026,8,17)),
            (a2,'Transfer',3500,'Multiple transfer review',date(2026,8,16))]
        for i,(acct,typ,amt,desc,dt) in enumerate(samples,1):
            db.session.add(Transaction(reference=f'TXN{i:04d}',account_id=acct.id,txn_type=typ,amount=amt,description=desc,txn_date=dt))
        db.session.commit()

def account_balance(account):
    total = account.opening_balance
    for t in account.transactions:
        if t.txn_type == 'Deposit': total += t.amount
        elif t.txn_type == 'Withdrawal': total -= t.amount
    return total

def money(n): return float(n or 0)

@app.route('/')
def dashboard():
    accounts = Account.query.order_by(Account.id).all()
    transactions = Transaction.query.order_by(Transaction.txn_date.desc(), Transaction.id.desc()).all()
    deposits = sum(t.amount for t in transactions if t.txn_type == 'Deposit')
    withdrawals = sum(t.amount for t in transactions if t.txn_type == 'Withdrawal')
    balance = sum(account_balance(a) for a in accounts)
    flagged = [t for t in transactions if t.is_flagged]
    months = {}
    for t in transactions:
        key=t.txn_date.strftime('%b %Y')
        months.setdefault(key, {'deposits':0,'withdrawals':0})
        if t.txn_type == 'Deposit': months[key]['deposits'] += t.amount
        if t.txn_type == 'Withdrawal': months[key]['withdrawals'] += t.amount
    month_items=sorted(months.items(), key=lambda x: datetime.strptime(x[0],'%b %Y'))[-6:]
    type_totals={k:sum(t.amount for t in transactions if t.txn_type==k) for k in ['Deposit','Withdrawal','Transfer']}
    return render_template('index.html', accounts=accounts, transactions=transactions[:10], all_transactions=transactions,
        deposits=deposits, withdrawals=withdrawals, balance=balance, flagged=flagged,
        month_labels=[x[0] for x in month_items], month_deposits=[x[1]['deposits'] for x in month_items],
        month_withdrawals=[x[1]['withdrawals'] for x in month_items], type_labels=list(type_totals), type_values=list(type_totals.values()),
        account_balance=account_balance)

@app.route('/transactions')
def transactions_page():
    q=request.args.get('q','').strip(); kind=request.args.get('type','')
    query=Transaction.query.join(Account)
    if q: query=query.filter(db.or_(Transaction.reference.ilike(f'%{q}%'),Transaction.description.ilike(f'%{q}%'),Account.name.ilike(f'%{q}%')))
    if kind in ['Deposit','Withdrawal','Transfer']: query=query.filter(Transaction.txn_type==kind)
    transactions=query.order_by(Transaction.txn_date.desc(),Transaction.id.desc()).all()
    return render_template('transactions.html', transactions=transactions, accounts=Account.query.all(), q=q, kind=kind)

@app.route('/transactions/add', methods=['POST'])
def add_transaction():
    try:
        account_id=int(request.form.get('account_id','0')); amount=float(request.form.get('amount','0'))
        kind=request.form.get('txn_type',''); desc=request.form.get('description','').strip()
        txn_date=datetime.strptime(request.form.get('txn_date',''),'%Y-%m-%d').date()
        if amount <= 0 or kind not in ['Deposit','Withdrawal','Transfer'] or not desc: raise ValueError('Enter a valid positive amount, type, description and date.')
        account=Account.query.get_or_404(account_id)
        if kind=='Withdrawal' and amount>account_balance(account): raise ValueError('Withdrawal exceeds the available simulated balance.')
        ref='TXN'+datetime.utcnow().strftime('%y%m%d%H%M%S%f')[-12:]
        db.session.add(Transaction(reference=ref,account_id=account.id,txn_type=kind,amount=amount,description=desc,txn_date=txn_date))
        db.session.commit(); flash('Transaction added successfully.','success')
    except (ValueError, InvalidOperation) as e:
        flash(str(e) or 'Please check the transaction details.','error')
    return redirect(request.referrer or url_for('dashboard'))

@app.route('/transactions/<int:txn_id>/delete', methods=['POST'])
def delete_transaction(txn_id):
    t=Transaction.query.get_or_404(txn_id); db.session.delete(t); db.session.commit();
```

```
        flash('Transaction deleted.','success')
    return redirect(request.referrer or url_for('transactions_page'))

@app.route('/accounts')
def accounts_page():
    accounts=Account.query.order_by(Account.id).all()
    return render_template('accounts.html', accounts=accounts, account_balance=account_balance)

@app.route('/accounts/add', methods=['POST'])
def add_account():
    try:
        name=request.form.get('name','').strip(); kind=request.form.get('account_type','Savings');
          opening=float(request.form.get('opening_balance','0'))
        if not name or opening < 0 or kind not in ['Savings','Checking','Fixed Deposit']: raise
          ValueError('Enter valid account details.')
        customer=Customer.query.first()
        account=Account(account_number='**** '+datetime.utcnow().strftime('%H%M%S'),name=name,account_ty
          pe=kind,opening_balance=opening,customer_id=customer.id)
        db.session.add(account); db.session.commit(); flash('Account created.','success')
    except ValueError as e: flash(str(e),'error')
    return redirect(url_for('accounts_page'))

@app.route('/analytics')
def analytics_page():
    return redirect(url_for('dashboard')+'#analytics')

@app.route('/health')
def health(): return jsonify(status='ok', framework='Flask')

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=int(os.getenv('PORT', 5000)), debug=True)
```

02 · api/index.py

Vercel serverless entry point

```
# Vercel Python/Flask entrypoint
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from app import app
```

03 · vercel.json

Vercel build and routing configuration

```
{
  "version": 2,
  "builds": [{ "src": "api/index.py", "use": "@vercel/python" }],
  "routes": [{ "src": "/(.*)", "dest": "api/index.py" }]
}
```

04 · requirements.txt

Python dependencies

```
Flask>=3.0,<4.0
Flask-SQLAlchemy>=3.1,<4.0
psycopg[binary]>=3.1,<4.0
```

05 · supabase/schema.sql

PostgreSQL schema for Supabase

```
-- Run this in the Supabase SQL Editor if you use Supabase Postgres.
-- Set DATABASE_URL in Vercel to your Supabase Postgres connection string (server-side only).
create table if not exists customers (
  id bigserial primary key,
  name text not null,
  email text not null unique,
  created_at timestamptz not null default now()
);
```

```sql
create table if not exists accounts (
  id bigserial primary key,
  account_number text not null unique,
  name text not null,
  account_type text not null check (account_type in ('Savings','Checking','Fixed Deposit')),
  opening_balance numeric(14,2) not null default 0 check (opening_balance >= 0),
  customer_id bigint not null references customers(id) on delete cascade,
  created_at timestamptz not null default now()
);
create table if not exists transactions (
  id bigserial primary key,
  reference text not null unique,
  account_id bigint not null references accounts(id) on delete cascade,
  txn_type text not null check (txn_type in ('Deposit','Withdrawal','Transfer')),
  amount numeric(14,2) not null check (amount > 0),
  description text not null,
  txn_date date not null default current_date,
  created_at timestamptz not null default now()
);
create index if not exists idx_transactions_date on transactions(txn_date desc);
create index if not exists idx_transactions_account on transactions(account_id);
```

06 · analysis/analyze_transactions.py

Optional pandas / NumPy / Matplotlib analysis script

```python
"""Optional analysis of the synthetic transaction CSV using Pandas, NumPy and Matplotlib."""
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

source = Path(__file__).with_name('sample_transactions.csv')
df = pd.read_csv(source, parse_dates=['date'])
print('Transaction count:', len(df))
print('Total amount:', round(float(np.sum(df['amount'])), 2))
print('\nTotals by type:')
print(df.groupby('type')['amount'].agg(['count','sum']).round(2))
df['review_flag'] = (df['amount'] >= 5000) | ((df['type'] == 'Withdrawal') & (df['amount'] >= 2000))
print('\nFlagged for manual review:')
print(df.loc[df.review_flag, ['reference','date','type','amount','description']].to_string(index=False))
monthly = df.assign(month=df['date'].dt.to_period('M').astype(str)).pivot_table(index='month',
  columns='type', values='amount', aggfunc='sum', fill_value=0)
monthly.plot(kind='bar', figsize=(9,5), title='Monthly Transaction Amounts')
plt.ylabel('Amount'); plt.tight_layout();
  plt.savefig(Path(__file__).with_name('monthly_transactions.png'), dpi=140)
print('\nSaved monthly_transactions.png')
```

07 · templates/base.html

Base layout: sidebar, top bar, flash messages (formatted with line breaks for readability)

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8"><meta name="viewport" content="width=device-width,
      initial-scale=1"><title>LedgerLens · Banking Analysis</title><link rel="stylesheet" href="{{
      url_for('static', filename='style.css') }}">
    <script src="https://cdn.jsdelivr.net/npm/chart.js">
    </script>
  </head>
  <body>
    <aside class="sidebar">
      <div class="brand">
        <span class="brand-icon">▦</span>
        <div>
          <b>LedgerLens</b><small>Banking Analysis</small>
        </div>
      </div>
      <nav>
        <a href="{{ url_for('dashboard') }}" class="{{ 'active' if request.endpoint=='dashboard' else ''
          }}">■   Dashboard</a><a href="{{ url_for('transactions_page') }}" class="{{ 'active' if
          request.endpoint=='transactions_page' else '' }}">⇄   Transactions</a><a href="{{
          url_for('accounts_page') }}" class="{{ 'active' if request.endpoint=='accounts_page' else ''
          }}">▣   Accounts</a><a href="{{ url_for('dashboard') }}#analytics">▤   Analytics</a>
      </nav>
```

```
      <div class="sidebar-bottom">
        <span class="avatar">B</span>
        <div>
          <b>Demo Customer</b><small>Local demo workspace</small>
        </div>
      </div>
    </aside>
    <main class="main">
      <header class="topbar">
        <div class="crumb">
          Banking / <b>
          {% block section %}
            Dashboard
          {% endblock %}
          </b>
        </div>
        <div class="top-right">
          <span class="status-dot"></span> Demo environment <span class="avatar small">B</span>
        </div>
      </header>
      <div class="content">
        {% with messages=get_flashed_messages(with_categories=true) %}
          {% if messages %}
            {% for category,message in messages %}
              <div class="flash {{ category }}">
                {{ message }}
              </div>
            {% endfor %}
          {% endif %}
        {% endwith %}
        {% block content %}
        {% endblock %}
        <footer>
          LedgerLens · Banking Transaction Analysis <span>Educational simulation · Synthetic data
            only</span>
        </footer>
      </div>
    </main>
    {% block scripts %}
    {% endblock %}
  </body>
</html>
```

08 · templates/index.html

Dashboard page with Chart.js charts (formatted with line breaks for readability)

```
{% extends 'base.html' %}
{% block content %}
  <section class="page-heading">
    <div>
      <p class="eyebrow">
        OVERVIEW
      </p>
      <h1>
        Good day, Demo Customer <span>✦</span>
      </h1>
      <p class="muted">
        Here’s what’s happening across your simulated accounts.
      </p>
    </div>
    <a class="button" href="{{ url_for('transactions_page') }}">＋ Add transaction</a>
  </section>
  <section class="stats">
    <article class="stat">
      <div class="stat-top">
        <span>Total deposits</span><span class="bubble green">↗</span>
      </div>
      <strong>₹{{ '{:,.2f}'.format(deposits) }}</strong><small class="positive">↑ Money in</small>
    </article>
    <article class="stat">
      <div class="stat-top">
        <span>Total withdrawals</span><span class="bubble red">↘</span>
      </div>
      <strong>₹{{ '{:,.2f}'.format(withdrawals) }}</strong><small class="negative">↓ Money out</small>
    </article>
    <article class="stat">
      <div class="stat-top">
```

```
        <span>Transactions</span><span class="bubble blue">⇄</span>
      </div>
      <strong>{{ all_transactions|length }}</strong><small class="muted">Across all accounts</small>
    </article>
    <article class="stat">
      <div class="stat-top">
        <span>Simulated balance</span><span class="bubble purple">▣</span>
      </div>
      <strong>₹{{ '{:,.2f}'.format(balance) }}</strong><small class="muted">Current calculated
        balance</small>
    </article>
  </section>
  <section class="grid two">
    <article class="panel" id="analytics">
      <div class="panel-heading">
        <div>
          <h2>
            Cash flow
          </h2>
          <p class="muted">
            Deposits and withdrawals by month
          </p>
        </div>
        <span class="pill">Monthly</span>
      </div>
      <div class="chart-wrap">
        <canvas id="cashFlow"></canvas>
      </div>
    </article>
    <article class="panel">
      <div class="panel-heading">
        <div>
          <h2>
            Transaction mix
          </h2>
          <p class="muted">
            Total amount by transaction type
          </p>
        </div>
      </div>
      <div class="chart-wrap doughnut">
        <canvas id="txnMix"></canvas>
      </div>
      <div class="legend-list">
        <span><i class="dot green-bg"></i> Deposits</span><span><i class="dot red-bg"></i>
          Withdrawals</span><span><i class="dot blue-bg"></i> Transfers</span>
      </div>
    </article>
  </section>
  <section class="grid two lower">
    <article class="panel">
      <div class="panel-heading">
        <div>
          <h2>
            Recent transactions
          </h2>
          <p class="muted">
            Your latest activity
          </p>
        </div>
        <a class="text-link" href="{{ url_for('transactions_page') }}">View all →</a>
      </div>
      <div class="table-scroll">
        <table>
          <thead>
            <tr>
              <th>Transaction</th><th>Type</th><th>Date</th><th class="right">Amount</th>
            </tr>
          </thead>
          <tbody>
            {% for t in transactions %}
              <tr>
                <td><b>{{ t.description }}</b><small>{{ t.reference }} · {{ t.account.name
                  }}</small></td><td><span class="tag {{ t.txn_type|lower }}">{{ t.txn_type
                  }}</span></td><td>{{ t.txn_date.strftime('%b %d, %Y') }}</td><td class="right
                  amount">{{ '+' if t.txn_type=='Deposit' else '-' if t.txn_type=='Withdrawal' else ''
                  }}₹{{ '{:,.2f}'.format(t.amount) }}</td>
              </tr>
            {% else %}
              <tr>
```

```
106                 <td colspan="4">No transactions yet.</td>
107               </tr>
108             {% endfor %}
109           </tbody>
110         </table>
111       </div>
112     </article>
113     <article class="panel">
114       <div class="panel-heading">
115         <div>
116           <h2>
117             Accounts
118           </h2>
119           <p class="muted">
120             Balances by account
121           </p>
122         </div>
123         <a class="text-link" href="{{ url_for('accounts_page') }}">Manage →</a>
124       </div>
125       <div class="account-list">
126         {% for a in accounts %}
127           <div class="account-row">
128             <span class="account-icon {{ loop.cycle('green','blue','purple') }}">▣</span>
129             <div class="account-info">
130               <b>{{ a.name }}</b><small>{{ a.account_type }} · {{ a.account_number }}</small>
131             </div>
132             <div class="account-amount">
133               <b>₹{{ '{:,.2f}'.format(account_balance(a)) }}</b><small>Calculated balance</small>
134             </div>
135           </div>
136         {% endfor %}
137       </div>
138       <div class="review-box">
139         <div class="review-icon">
140           !
141         </div>
142         <div>
143           <b>{{ flagged|length }} transactions flagged for review</b>
144           <p>
145             Rule-based checks highlight large or unusual amounts.
146           </p>
147         </div>
148       </div>
149     </article>
150   </section>
151   <section class="panel flags-panel">
152     <div class="panel-heading">
153       <div>
154         <h2>
155           Unusual transaction review
156         </h2>
157         <p class="muted">
158           Rule-based review flags — not a fraud determination
159         </p>
160       </div>
161       <span class="pill warn">{{ flagged|length }} to review</span>
162     </div>
163     {% if flagged %}
164       <div class="table-scroll">
165         <table>
166           <thead>
167             <tr>
168               <th>Reference</th><th>Account</th><th>Reason</th><th
                    class="right">Amount</th><th>Status</th>
169             </tr>
170           </thead>
171           <tbody>
172             {% for t in flagged %}
173               <tr>
174                 <td><b>{{ t.reference }}</b><small>{{ t.txn_date.strftime('%b %d, %Y')
                      }}</small></td><td>{{ t.account.name }}</td><td>{{ t.flag_reason }}</td><td
                      class="right">₹{{ '{:,.2f}'.format(t.amount) }}</td><td><span class="tag
                      pending">Needs review</span></td>
175               </tr>
176             {% endfor %}
177           </tbody>
178         </table>
179       </div>
180     {% else %}
181       <p class="empty">
```

```
        No transactions currently match the review rules.
      </p>
    {% endif %}
  </section>
{% endblock %}
{% block scripts %}
  <script>
    const labels={{ month_labels|tojson }};new
      Chart(document.getElementById('cashFlow'),{type:'line',data:{labels,datasets:[{label:'Deposits',da
      ta:{{ month_deposits|tojson }},borderColor:'#21b78b',backgroundColor:'rgba(33,183,139,.1)',tension
      :.35,fill:true},{label:'Withdrawals',data:{{ month_withdrawals|tojson }},borderColor:'#f45b75',bac
      kgroundColor:'rgba(244,91,117,.05)',tension:.35}]},options:{responsive:true,maintainAspectRatio:fa
      lse,plugins:{legend:{position:'bottom'}},scales:{y:{beginAtZero:true,grid:{color:'#edf1f7'}},x:{gr
      id:{display:false}}}});new
      Chart(document.getElementById('txnMix'),{type:'doughnut',data:{labels:{{ type_labels|tojson
      }},datasets:[{data:{{ type_values|tojson }},backgroundColor:['#21b78b','#f45b75','#4c8df6'],border
      Width:0}]},options:{responsive:true,maintainAspectRatio:false,cutout:'68%',plugins:{legend:{positi
      on:'bottom'}}}});
  </script>
{% endblock %}
```

09 · templates/transactions.html

Transactions list, search/filter and add form (formatted with line breaks for readability)

```
{% extends 'base.html' %}
{% block section %}
  Transactions
{% endblock %}
{% block content %}
  <section class="page-heading">
    <div>
      <p class="eyebrow">
        ACTIVITY
      </p>
      <h1>
        Transactions
      </h1>
      <p class="muted">
        Review and manage simulated transaction history.
      </p>
    </div>
    <button class="button" onclick="document.getElementById('addForm').classList.toggle('hidden')">＋ Add
       transaction</button>
  </section>
  <form class="panel add-form hidden" id="addForm" method="post" action="{{ url_for('add_transaction')
    }}">
    <h2>
      New transaction
    </h2>
    <div class="form-grid">
      <label>
        Account<select name="account_id" required>
        {% for a in accounts %}
          <option value="{{ a.id }}">{{ a.name }} ({{ a.account_number }})</option>
        {% endfor %}
        </select>
      </label>
      <label>
        Type<select
          name="txn_type"><option>Deposit</option><option>Withdrawal</option><option>Transfer</option></
          select>
      </label>
      <label>
        Amount (₹)<input type="number" name="amount" min="0.01" step="0.01" required>
      </label>
      <label>
        Date<input type="date" name="txn_date" value="{{ now or '' }}" required>
      </label>
      <label class="full">
        Description<input name="description" maxlength="240" required placeholder="e.g. Salary credit">
      </label>
    </div>
    <button class="button" type="submit">Save transaction</button>
  </form>
  <section class="panel">
    <form class="filters">
      <input name="q" value="{{ q }}" placeholder="Search reference, description, account…"><select
```

```
          name="type"><option value="">All transaction types</option>
        {% for x in ['Deposit','Withdrawal','Transfer'] %}
          <option {{ 'selected' if kind==x else '' }}>{{ x }}</option>
        {% endfor %}
        </select><button class="button secondary">Filter</button>
      </form>
      <div class="table-scroll">
        <table>
          <thead>
            <tr>
              <th>Reference</th><th>Date</th><th>Account</th><th>Type</th><th>Description</th><th
                class="right">Amount</th><th></th>
            </tr>
          </thead>
          <tbody>
            {% for t in transactions %}
              <tr>
                <td><b>{{ t.reference }}</b></td><td>{{ t.txn_date.strftime('%b %d, %Y') }}</td><td>{{
                  t.account.name }}</td><td><span class="tag {{ t.txn_type|lower }}">{{ t.txn_type
                  }}</span></td><td>{{ t.description }}
                {% if t.is_flagged %}
                  <small class="warning-text">Review: {{ t.flag_reason }}</small>
                {% endif %}
                </td><td class="right">₹{{ '{:,.2f}'.format(t.amount) }}</td><td>
                <form method="post" action="{{ url_for('delete_transaction',txn_id=t.id) }}"
                  onsubmit="return confirm('Delete this transaction?')">
                  <button class="icon-button" title="Delete">×</button>
                </form>
                </td>
              </tr>
            {% else %}
              <tr>
                <td colspan="7" class="empty">No matching transactions.</td>
              </tr>
            {% endfor %}
          </tbody>
        </table>
      </div>
      <p class="table-foot">
        {{ transactions|length }} transaction(s) found · Synthetic data only
      </p>
    </section>
{% endblock %}
```

10 · templates/accounts.html

Account cards and create-account form (formatted with line breaks for readability)

```
{% extends 'base.html' %}
{% block section %}
  Accounts
{% endblock %}
{% block content %}
  <section class="page-heading">
    <div>
      <p class="eyebrow">
        YOUR MONEY
      </p>
      <h1>
        Account summary
      </h1>
      <p class="muted">
        Balances are calculated from opening balances and recorded activity.
      </p>
    </div>
    <button class="button" onclick="document.getElementById('accountForm').classList.toggle('hidden')">
       Add account</button>
  </section>
  <form class="panel add-form hidden" id="accountForm" method="post" action="{{ url_for('add_account')
    }}">
    <h2>
      Create account
    </h2>
    <div class="form-grid">
      <label>
        Account name<input name="name" required placeholder="e.g. Travel savings">
      </label>
      <label>
```

```
        Account type<select
          name="account_type"><option>Savings</option><option>Checking</option><option>Fixed
          Deposit</option></select>
      </label>
      <label>
        Opening balance (₹)<input type="number" name="opening_balance" min="0" step="0.01" value="0"
          required>
      </label>
    </div>
    <button class="button">Create account</button>
  </form>
  <section class="account-cards">
    {% for a in accounts %}
      <article class="panel account-card">
        <span class="account-icon {{ loop.cycle('green','blue','purple') }}">▣</span><span
          class="pill">Active · Demo</span>
        <h2>
          {{ a.name }}
        </h2>
        <p class="muted">
          {{ a.account_type }} account
        </p>
        <div class="account-number">
          {{ a.account_number }}
        </div>
        <p class="muted small-label">
          Available simulated balance
        </p>
        <strong>₹{{ '{:,.2f}'.format(account_balance(a)) }}</strong>
        <div class="account-card-foot">
          <span>Opening balance</span><b>₹{{ '{:,.2f}'.format(a.opening_balance) }}</b>
        </div>
        <div class="account-card-foot">
          <span>Transactions</span><b>{{ a.transactions|length }}</b>
        </div>
      </article>
    {% endfor %}
  </section>
  <p class="note">
    These accounts and balances are synthetic examples for an educational project. They do not connect
      to real bank accounts.
  </p>
{% endblock %}
```

11 · static/style.css

Stylesheet (formatted with line breaks for readability)

```
:root{
  --navy:#172554;
  --navy2:#312e81;
  --blue:#5b5cf0;
  --text:#17253b;
  --muted:#7c899c;
  --line:#e7edf5;
  --bg:#f5f6ff;
  --green:#10b981;
  --red:#f43f5e}
*{
  box-sizing:border-box}
body{
  margin:0;
  background:radial-gradient(ellipse at 85% 0%,#e7eaff 0,transparent 34%),var(--bg);
  color:var(--text);
  font-family:Inter,ui-sans-serif,system-ui,-apple-system,"Segoe UI",sans-serif;
  font-size:14px}
.sidebar{
  position:fixed;
  inset:0 auto 0 0;
  width:238px;
  background:linear-gradient(160deg,#111c4e 0%,#312e81 55%,#4c1d95 100%);
  color:#fff;
  padding:25px 15px;
  display:flex;
  flex-direction:column;
  z-index:5}
.brand{
  display:flex;
```

```
  align-items:center;
  gap:12px;
  padding:4px 12px 35px}
.brand-icon{
  display:grid;
  place-items:center;
  width:38px;
  height:38px;
  border-radius:12px;
  background:#2672f2;
  font-size:23px}
.brand b{
  display:block;
  font-size:15px;
  letter-spacing:.1px}
.brand small,.sidebar-bottom small{
  display:block;
  color:#a8b7ce;
  font-size:11px;
  margin-top:4px}
.sidebar nav{
  display:grid;
  gap:7px}
.sidebar nav a{
  color:#b9c6d8;
  text-decoration:none;
  padding:13px 14px;
  border-radius:9px;
  font-weight:600}
.sidebar nav a:hover,.sidebar nav a.active{
  color:white;
  background:linear-gradient(100deg,#6366f1,#8b5cf6);
  box-shadow:0 7px 20px #6366f144}
.sidebar-bottom{
  margin-top:auto;
  border-top:1px solid #34465e;
  padding:20px 8px 0;
  display:flex;
  align-items:center;
  gap:10px}
.avatar{
  width:35px;
  height:35px;
  display:grid;
  place-items:center;
  border-radius:50%;
  background:#e3edff;
  color:#1b56ae;
  font-weight:800}
.avatar.small{
  width:28px;
  height:28px;
  font-size:12px}
.main{
  margin-left:238px;
  min-height:100vh}
.topbar{
  height:68px;
  background:rgba(255,255,255,.88);
  backdrop-filter:blur(14px);
  border-bottom:1px solid var(--line);
  display:flex;
  align-items:center;
  justify-content:space-between;
  padding:0 36px}
.crumb{
  color:#8a96a8}
.crumb b{
  color:#23334b}
.top-right{
  display:flex;
  align-items:center;
  gap:10px;
  color:#65758b;
  font-size:12px}
.status-dot{
  width:8px;
  height:8px;
  border-radius:50%;
  background:#20b789}
```

```
.content{
  max-width:1600px;
  margin:auto;
  padding:31px 36px 20px}
.page-heading{
  display:flex;
  justify-content:space-between;
  align-items:center;
  margin-bottom:24px;
  gap:20px}
.eyebrow{
  color:#276dec;
  font-size:10px;
  letter-spacing:1.4px;
  font-weight:800;
  margin:0 0 8px}
.page-heading h1{
  font-size:29px;
  letter-spacing:-.8px;
  margin:0 0 7px;
  background:linear-gradient(90deg,#172554,#5b21b6);
  -webkit-background-clip:text;
  background-clip:text;
  color:transparent}
.page-heading h1 span{
  color:#eab54e;
  font-size:19px}
.muted{
  color:var(--muted);
  font-size:12px;
  margin:0}
.button{
  border:0;
  border-radius:8px;
  background:linear-gradient(110deg,#4f46e5,#7c3aed);
  color:#fff;
  padding:11px 16px;
  font-weight:700;
  font-size:12px;
  text-decoration:none;
  cursor:pointer;
  box-shadow:0 7px 18px #6366f044;
  transition:transform .18s,box-shadow .18s;
  white-space:nowrap}
.button:hover{
  background:linear-gradient(110deg,#4338ca,#6d28d9);
  transform:translateY(-1px);
  box-shadow:0 9px 22px #6366f055}
.button.secondary{
  background:#edf3ff;
  color:#275bc2;
  box-shadow:none}
.stats{
  display:grid;
  grid-template-columns:repeat(4,minmax(0,1fr));
  gap:17px;
  margin-bottom:20px}
.stat,.panel{
  background:#fff;
  border:1px solid var(--line);
  border-radius:16px;
  box-shadow:0 8px 28px #3b4a8a0b;
  transition:transform .18s,box-shadow .18s}
.stat{
  padding:20px;
  border-top:3px solid #c7d2fe}
.stat:nth-child(2){
  border-top-color:#fda4af}
.stat:nth-child(3){
  border-top-color:#7dd3fc}
.stat:nth-child(4){
  border-top-color:#c4b5fd}
.stat:hover,.account-card:hover{
  transform:translateY(-3px);
  box-shadow:0 14px 32px #3b4a8a18}
.stat-top{
  display:flex;
  align-items:center;
  justify-content:space-between;
  color:#77869a;
```

```
    font-size:12px;
    margin-bottom:17px}
  .bubble{
    width:34px;
    height:34px;
    display:grid;
    place-items:center;
    border-radius:11px;
    font-size:16px;
    font-weight:800}
  .green{
    background:#e1f8f0;
    color:#15a77a}
  .red{
    background:#ffebef;
    color:#e84f6d}
  .blue{
    background:#e8f1ff;
    color:#3277ed}
  .purple{
    background:#f1e9ff;
    color:#8958db}
  .stat strong{
    display:block;
    font-size:25px;
    letter-spacing:-.5px;
    margin-bottom:9px}
  .positive,.negative{
    font-size:11px;
    color:#17a779}
  .negative{
    color:#e35d74}
  .stat small.muted{
    font-size:11px}
  .grid{
    display:grid;
    gap:18px}
  .grid.two{
    grid-template-columns:1.15fr 1fr}
  .panel{
    padding:21px;
    margin-bottom:19px;
    min-width:0}
  .panel-heading{
    display:flex;
    align-items:center;
    justify-content:space-between;
    gap:12px;
    margin-bottom:17px}
  .panel-heading h2,.add-form h2{
    font-size:15px;
    margin:0 0 6px}
  .panel-heading p{
    font-size:11px}
  .pill{
    background:#edf3ff;
    color:#426fc0;
    border-radius:20px;
    padding:6px 10px;
    font-size:10px;
    font-weight:700;
    white-space:nowrap}
  .pill.warn{
    background:#fff4df;
    color:#b87916}
  .chart-wrap{
    height:235px;
    position:relative}
  .chart-wrap.doughnut{
    height:205px;
    max-width:340px;
    margin:auto}
  .legend-list{
    display:flex;
    justify-content:center;
    gap:20px;
    font-size:10px;
    color:#6f7d90;
    margin-top:5px}
  .dot{
```

```
  display:inline-block;
  width:7px;
  height:7px;
  border-radius:50%;
  margin-right:5px}
.green-bg{
  background:#20b789}
.red-bg{
  background:#ef5b75}
.blue-bg{
  background:#4c8df6}
.lower{
  align-items:start}
.text-link{
  font-size:11px;
  font-weight:700;
  text-decoration:none;
  color:#246cf0;
  white-space:nowrap}
.table-scroll{
  overflow-x:auto}
table{
  width:100%;
  border-collapse:collapse;
  text-align:left;
  font-size:11px;
  white-space:nowrap}
th{
  color:#8a96a8;
  font-size:10px;
  font-weight:700;
  background:linear-gradient(90deg,#f7f8ff,#fbfcff);
  padding:12px 10px;
  border-bottom:1px solid var(--line)}
td{
  padding:13px 10px;
  border-bottom:1px solid #eef2f7;
  color:#4d5c70}
td b{
  color:#26364e;
  font-weight:700}
td small{
  display:block;
  font-size:9px;
  color:#94a0b1;
  margin-top:4px}
.right{
  text-align:right}
.amount{
  font-weight:800;
  color:#243650}
.tag{
  display:inline-block;
  padding:5px 8px;
  border-radius:20px;
  font-size:9px;
  font-weight:800}
.tag.deposit{
  background:#e1f8f0;
  color:#12986d}
.tag.withdrawal{
  background:#ffebef;
  color:#e34e6c}
.tag.transfer{
  background:#e8f1ff;
  color:#3977e5}
.tag.pending{
  background:#fff0d8;
  color:#b87816}
.account-list{
  display:grid;
  gap:0}
.account-row{
  display:flex;
  align-items:center;
  gap:11px;
  padding:13px 0;
  border-bottom:1px solid #eef2f7}
.account-icon{
  width:36px;
```

```
  height:36px;
  border-radius:11px;
  display:grid;
  place-items:center;
  flex-shrink:0}
.account-info{
  min-width:0;
  flex:1}
.account-info b{
  font-size:11px;
  display:block}
.account-info small,.account-amount small{
  font-size:9px;
  color:#8c99aa;
  display:block;
  margin-top:5px}
.account-amount{
  text-align:right}
.account-amount b{
  font-size:12px}
.review-box{
  display:flex;
  align-items:center;
  gap:11px;
  background:#fff8e9;
  border:1px solid #ffedc5;
  border-radius:10px;
  padding:14px;
  margin-top:18px}
.review-icon{
  width:28px;
  height:28px;
  display:grid;
  place-items:center;
  border-radius:50%;
  background:#ffebc5;
  color:#b87916;
  font-weight:900}
.review-box b{
  font-size:11px}
.review-box p{
  font-size:10px;
  color:#8c7b5e;
  margin:5px 0 0}
.flags-panel{
  margin-top:1px}
.empty{
  padding:25px;
  color:#8794a6;
  text-align:center}
.table-foot{
  color:#9aa5b5;
  font-size:10px;
  margin:15px 0 0}
.filters{
  display:flex;
  gap:10px;
  margin-bottom:17px}
.filters input{
  flex:1}
.filters input,.filters select,input,select{
  border:1px solid #dfe6f0;
  border-radius:7px;
  background:#fff;
  padding:11px 12px;
  color:#34455e;
  font:inherit;
  font-size:12px;
  min-width:0}
.filters select{
  min-width:180px}
.hidden{
  display:none}
.add-form{
  background:#fbfdff}
.form-grid{
  display:grid;
  grid-template-columns:repeat(2,minmax(0,1fr));
  gap:15px;
  margin:18px 0}
```

```
.form-grid label{
  display:grid;
  gap:7px;
  font-size:11px;
  color:#6b7b90;
  font-weight:700}
.form-grid .full{
  grid-column:1/-1}
.icon-button{
  border:0;
  background:#fff1f2;
  color:#d94d67;
  border-radius:6px;
  width:26px;
  height:26px;
  cursor:pointer;
  font-size:18px}
.account-cards{
  display:grid;
  grid-template-columns:repeat(3,minmax(0,1fr));
  gap:18px}
.account-card{
  padding:22px}
.account-card .pill{
  float:right}
.account-card h2{
  font-size:15px;
  margin:20px 0 6px}
.account-number{
  font-size:11px;
  color:#8a96a8;
  margin-top:17px}
.small-label{
  margin-top:18px}
.account-card>strong{
  font-size:26px;
  display:block;
  margin:7px 0 22px}
.account-card-foot{
  border-top:1px solid #edf1f6;
  padding:12px 0 0;
  margin-top:11px;
  display:flex;
  justify-content:space-between;
  color:#8794a6;
  font-size:11px}
.account-card-foot b{
  color:#33445d}
.note{
  font-size:11px;
  color:#8794a6}
.warning-text{
  color:#c47e13!important}
.flash{
  padding:12px 15px;
  border-radius:8px;
  margin-bottom:15px;
  font-size:12px}
.flash.success{
  background:#e1f8f0;
  color:#087d58}
.flash.error{
  background:#fff0f0;
  color:#b53349}
footer{
  display:flex;
  justify-content:space-between;
  color:#9aa5b5;
  font-size:10px;
  padding:12px 2px 0}
footer span{
  color:#a7b0bd}
@media(max-width:1100px){
  .sidebar{
  width:205px}
.main{
  margin-left:205px}
.content{
  padding:25px 22px}
.stats{
```

```
  grid-template-columns:repeat(2,minmax(0,1fr))}
.grid.two{
  grid-template-columns:1fr}
.account-cards{
  grid-template-columns:1fr 1fr}
}
@media(max-width:700px){
  .sidebar{
  position:static;
  width:auto;
  height:auto;
  padding:12px}
.brand{
  padding:4px 7px 12px}
.sidebar nav{
  grid-template-columns:repeat(4,minmax(0,1fr));
  gap:4px}
.sidebar nav a{
  text-align:center;
  padding:10px 4px;
  font-size:10px}
.sidebar-bottom{
  display:none}
.main{
  margin-left:0}
.topbar{
  height:48px;
  padding:0 15px}
.content{
  padding:20px 13px}
.page-heading{
  align-items:flex-start;
  flex-direction:column}
.page-heading h1{
  font-size:23px}
.stats{
  gap:10px}
.stat{
  padding:14px}
.stat strong{
  font-size:20px}
.stat-top{
  font-size:11px;
  margin-bottom:10px}
.panel{
  padding:14px}
.account-cards{
  grid-template-columns:1fr}
.filters{
  flex-wrap:wrap}
.filters input{
  flex-basis:100%}
.filters select{
  flex:1;
  min-width:0}
.form-grid{
  grid-template-columns:1fr}
.form-grid .full{
  grid-column:auto}
.top-right{
  font-size:10px}
footer{
  gap:10px;
  flex-direction:column}
}
```